EEM 364
Mikroişlemcileri

```
; dizi icerisinde en yuksek degeri bulmak
.model small

.data

 x    db 12,23,2,4,8,1,2,32,14,76,99,14,53,32,44
 lx  equ $-x
 MX   db 0

.code
 mov ax , @data
 mov ds , ax

 mov ax , 0
 mov si , 0

 L:
 mov al , x[si]
 cmp MX , al
 jle degis
 D:
 inc si
 cmp si , lx
 jnz L


HLT

degis:
mov MX , al
jmp D
```

```
; palindrom tespiti
.model small

.data

 x    db 1,2,3,4,0,3,2,1
 lx  equ ($-x)
 lh  equ lx/2

 F    db 1   ; sizi ilk olarak palindrom
             ; olarak tanimlanir

.code
 mov ax , @data
 mov ds , ax

 mov ax , 0
 mov si , 0
 mov di , lx-1

 L:
 mov al , x[si]
 mov ah , x[di]

 cmp al , ah
 jnz D

 dec di
 inc si
 cmp si , lh
 jnz L


SON:


HLT

D:
mov F , 0
jmp SON
```

İç İçe Döngüler

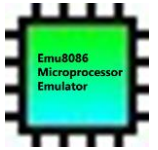
Diagonal Toplam

x db - - - - -

4 8 6 5
3 2 1 1
6 6 0 1
2 1 0 0

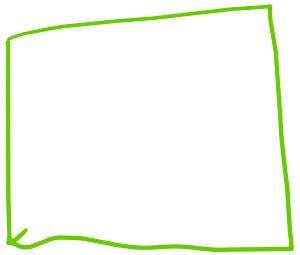
mov al, x[si]
cmp al, dh
jz TPL

TPL:
add T, al



İç İçe Döngüler

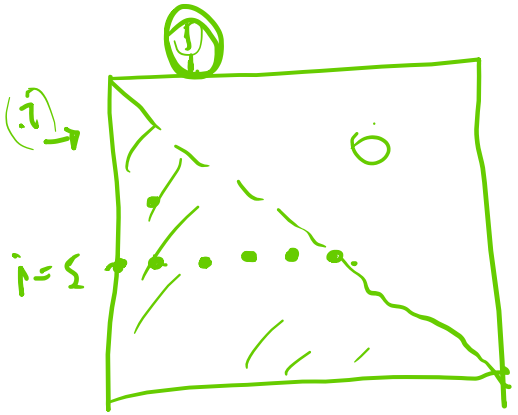
Matris içinde belli bir elemanı sayma



```
mov al, x[si]
cmp al, K
je CNT
D:
;

HLT

CNT:
inc di
jmp D
```

İç İçe Döngüler

Alt Üçgen Toplam



db

dh = satır
dl = sütun

mov al, x[si]

cmp dh, dl
jge T.

T.

Kesmeler

DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

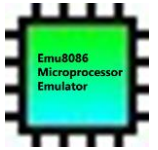
AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

```
.model small

.data

    msg db "Bir sayi giriniz:  $"

.code

main proc far
    mov ax , @data
    mov ds ,ax

    mov dx, offset msg

    mov  ah, 9
    int  21H


    mov ah , 01H
    int  21H


    mov  ah, 4CH  ; HLT YERINE
    int  21H
main endp
```

Kesmeler

5 sayı girme

5 sayı toplama



Kesmeler

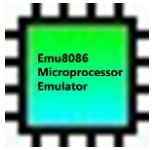
Yeni Satır

```
01 .model small
02
03 .data
04
05     msg1 db "5 sayi giriniz:   $"
06     msg2 db "2 sayi giriniz:   $"
07
08     ys db 0AH , 0DH , '$'
09
10 .code
11
12 main proc far
13     mov ax , @data
14     mov ds ,ax
15
16     lea  dx, msg1
17     mov  ah, 9
18     int  21H
19
20
21     lea  dx, ys
22     mov  ah, 9
23     int  21H
24
25
26     lea  dx, msg2
27     mov  ah, 9
28     int  21H
29
30     hlt
31
32 main endp
```

Kesmeler

«s» Karakter kontrolü

«s» gelinceye kadar
«a» sayma



Makrolar

emu8086.inc defines the following **macros**:

- **PRINT string** - macro with 1 parameter, prints out a string.
- **PRINTN string** - macro with 1 parameter, prints out a string. The same as PRINT but automatically adds "carriage return" at the end of the string.
- **PUTC char** - macro with 1 parameter, prints out an ASCII char at current cursor position.
- **GOTOXY col, row** - macro with 2 parameters, sets cursor position.
- **CURSOROFF** - turns off the text cursor.
- **CURSORON** - turns on the text cursor.

Makrolar

```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8


    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'


    hlt


main endp
```

Makrolar

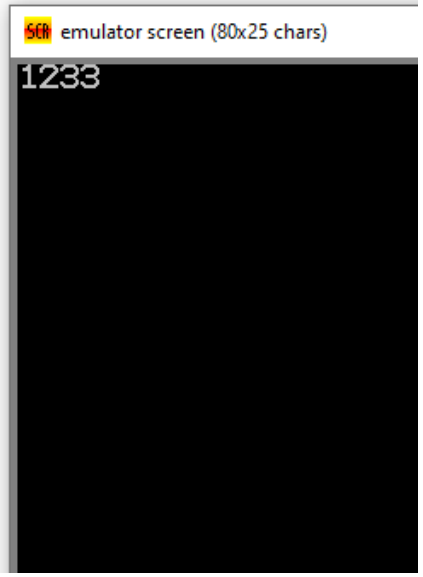
```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8


    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'

    hlt

main endp
```

```
; sets current cursor
; position:
GOTOXY  MACRO   col, row
        PUSH    AX
        PUSH    BX
        PUSH    DX
        MOV     AH, 02h
        MOV     DH, row
        MOV     DL, col
        MOV     BH, 0
        INT     10h
        POP     DX
        POP     BX
        POP     AX
ENDM
```

Makrolar

```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8

    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'

    hlt

main endp
```

```
.model small
.data
.code
main proc far

mov ax , 1233
call print_num_uns


main endp

DEFINE_PRINT_NUM_UNS
```



Makrolar

emu8086.inc defines the following **macros**:

- **PRINT string** - macro with 1 parameter, prints out a string.
- **PRINTN string** - macro with 1 parameter, prints out a string. The same as PRINT but automatically adds "carriage return" at the end of the string.
- **PUTC char** - macro with 1 parameter, prints out an ASCII char at current cursor position.
- **GOTOXY col, row** - macro with 2 parameters, sets cursor position.
- **CURSOROFF** - turns off the text cursor.
- **CURSORON** - turns on the text cursor.

- **CLEAR_SCREEN** - procedure to clear the screen, (done by scrolling entire screen window), and set cursor position to top of it. To use it declare: **DEFINE_CLEAR_SCREEN** before **END** directive.
- **PTHIS** - procedure to print a null terminated string at current cursor position (just as PRINT_STRING). The ZERO TERMINATED string should be defined just after the CALL. For example:

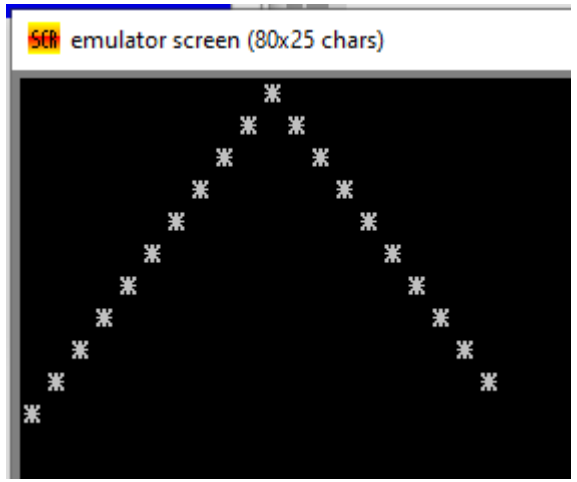
```
CALL PTHIS
db 'Hello World!', 0
```

  Address of string is stored in the Stack as return address. Procedure updates value in the Stack to make return after string definition. To use it declare: **DEFINE_PTHIS** before **END** directive.

- **SCAN_NUM** - procedure that gets the multi-digit SIGNED number from the keyboard, and stores the result in **CX** register. To use it declare: **DEFINE_SCAN_NUM** before **END** directive.

Makrolar

Şekil oluşturma



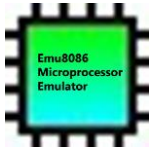

```
gotoxy ch , cl
putc '*'
inc ch
dec cl
```

Makrolar

Şekil oluşturma

Makrolar

Dışarıya tek karakter basma

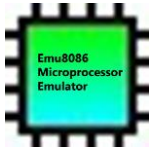


Makrolar

Matris Çarpması

Makrolar

Yeni makrolar ve prosedürler



Assembler Editor Plus, версия 0.9.5 (для тестов)

Файл Правка Вид Проект Запуск Модули Импорт Настройка Справка

@.ITEM .EQU

Открытые файлы:

- gpio.inc
- isr.asm
- main.asm

Каталог проекта

- LedBlink.aep
  - compile
  - modules
  - src
    - inc
    - isr.asm
    - link.ld
    - main.asm
  - tmp
  - LedBlink.aep

F1 Описание | F2 Файл | F3 Определение | F4 Изменить | F5 Символы | F6 Назад

```
.EQU  GPIOD              , PERIPH_BASE + GPIOD_BASE
.EQU  GPIOE              , PERIPH_BASE + GPIOE_BASE
.EQU  AFIO_BASE          , APB2PERIPH_BASE

@.ITEM     GPIOA GPIOB GPIOC GPIOD GPIOE GPIOA_BASE GPIOB_BASE GPIOC_BASE GPIOD_BASE GPIOE_BASE
.EQU  GPIO_CRL           , 0x00                  @ Port configuration register low (GPIOx_CRL)
.EQU  GPIO_CRH           , 0x04                  @ Port configuration register high (GPIOx_CRH)
.EQU  GPIO_IDR           , 0x08                  @ Port input data register (GPIOx_IDR)
.EQU  GPIO_ODR           , 0x0C                  @ Port output data register (GPIOx_ODR)
.EQU  GPIO_BSRR          , 0x10                  @ Port bi t se t/reset register (GPIOx_BSRR)
.EQU  GPIO_BRR           , 0x14                  @ Port bit reset register (GPIOx_BRR)
.EQU  GPIO_LCKR          , 0x18                  @ Port configuration lock register (GPIOx_LCKR)

@.ITEM     AFIO_BASE
.EQU  AFIO_EVCR          , 0x00                  @ Event co ntrol re gister (AFIO_EVCR)
.EQU  AFIO_MAPR          , 0x04                  @ AF remap and debug I/O configuration register (AFIO_MAPR)
.EQU  AFIO_EXTICR1       , 0x08                  @ External interrupt configuration register 1 (AFIO_EXTICR1)
.EQU  AFIO_EXTICR2       , 0x0C                  @ External interrupt configuration register 2 (AFIO_EXTICR2)
.EQU  AFIO_EXTICR3       , 0x10                  @ External interrupt configuration register 3 (AFIO_EXTICR3)
.EQU  AFIO_EXTICR4       , 0x14                  @ External interrupt configuration register 4 (AFIO_EXTICR4)
.EQU  AFIO_MAPR2         , 0x1C                  @ AF remap and debug I/O configuration register (AFIO_MAPR2)

@.ITEM     GPIO_CRL
.EQU  GPIO_CRL_RSTVALUE  , 0x44444444            @ значение регистра по умолчанию
.EQU  GPIO_CRL_MODE0_N   , 0                     @ 0:1  Pin 0 Port mode bits
.EQU  GPIO_CRL_MODE1_N   , 4                     @ 4:5  Pin 1 Port mode bits
.EQU  GPIO_CRL_MODE2_N   , 8                     @ 8:9  Pin 2 Port mode bits
.EQU  GPIO_CRL_MODE3_N   , 12                    @ 12:13  Pin 3 Port mode bits
.EQU  GPIO_CRL_MODE4_N   , 16                    @ 16:17  Pin 4 Port mode bits
.EQU  GPIO_CRL_MODE5_N   , 20                    @ 20:21  Pin 5 Port mode bits
.EQU  GPIO_CRL_MODE6_N   , 24                    @ 24:25  Pin 6 Port mode bits
.EQU  GPIO_CRL_MODE7_N   , 28                    @ 28:29  Pin 7 Port mode bits
.EQU  GPIO_CRL_CNF0_N    , 2                     @ 2:3  Pin 0 Port configuration bits
.EQU  GPIO_CRL_CNF1_N    , 6                     @ 6:7  Pin 1 Port configuration bits
.EQU  GPIO_CRL_CNF2_N    , 10                    @ 10:11  Pin 2 Port configuration bits
```

X: 31 : Y: 24 (200) BCT CP1251 d:\armasmedit\examples\stm32f1x\stm32v1discovery\ledblink\src\inc\gpio.inc

Assembler Editor Plus

Файл Правка Вид Проект Запуск Модули Настройки Справка

Открытые файлы:

- gpio.inc
- gpio_h_conf.inc
- rcc_ahb1enr_gpio_set.inc
- sysclk.asm
- link.ld
- main.asm
- sys.sasm

Каталог проекта

- test8.aep
  - bin
  - compile
    - internal
    - temp
    - symbols.lst
    - sys.bin
    - sys.dasm
    - sys.elf
    - sys.hex
    - sys.sasm
  - modules
  - src
  - tmp
  - make_project.bat
  - test8.aep

```
 8000098:    000000f0     .word    0x000000f0

080009c <IRQ_SysTick>:
 800009c:    b503         push     {r0, r1, lr}
 800009e:    490f         ldr      r1, [pc, #60]      ; (8000
 80000a0:    6808         ldr      r0, [r1, #0]
 80000a2:    f050 0000    orrs.w     r0, r0, #0
 80000a6:    bf1c         itt      ne
 80000a8:    3801         subne      r0, #1
 80000aa:    6008         strne      r0, [r1, #0]
 80000ac:    bd03         pop      {r0, r1, pc}

080000ae <SYSTICK_START>:
 80000ae:    b503         push     {r0, r1, lr}
 80000b0:    480b         ldr      r0, [pc, #44]      ; (8000
 80000b2:    490c         ldr      r1, [pc, #48]      ; (8000
 80000b4:    6041         str      r1, [r0, #4]
 80000b6:    490c         ldr      r1, [pc, #48]      ; (8000
 80000b8:    6001         str      r1, [r0, #0]
 80000ba:    480c         ldr      r0, [pc, #48]      ; (8000
 80000bc:    6a01         ldr      r1, [r0, #32]
 80000be:    f441 4170    orr.w      r1, r1, #61440      ; 0x
 80000c2:    6201         str      r1, [r0, #32]
 80000c4:    bd03         pop      {r0, r1, pc}

080000c6 <SYSTICK_DELAY>:
 80000c6:    b506         push     {r1, r2, lr}
 80000c8:    4a05         ldr      r2, [pc, #20]      ; (8000
 80000ca:    6090         str      r0, [r2, #8]
 80000cc:    4903         ldr      r1, [pc, #12]      ; (8000
 80000ce:    6008         str      r0, [r1, #0]

080000d0 <DELAY_LOOP>:
 80000d0:    6808         ldr      r0, [r1, #0]
 80000d2:    f050 0000    orrs.w     r0, r0, #0
 80000d6:    d1fb         bne.n    80000d0 <DELAY_LOOP>
 80000d8:    bd06         pop      {r1, r2, pc}
 80000da:    0000         .short     0x0000
 80000dc:    20000000     .word    0x20000000
```

X: 6 : Y: 29 (180) BCT UTF8 d:\workarea\test81\compile\sys.sasm

OpenODC сервер: Исполнение и Отладка

RUN | HALT | STATUS | RESET HALT | STEP | RESUME | RUN STEP | 1 ms | STOP

OpenODC Server | Telnet | MCU

Состояние: Privileged Halted

Инструкция: 80000d0: 6808 ldr r0, [r1, #0]

APSR: N Z C V Q GE: 0000 IPSR: 000000000 EPSR: ICI: 00 IT: 00000 T

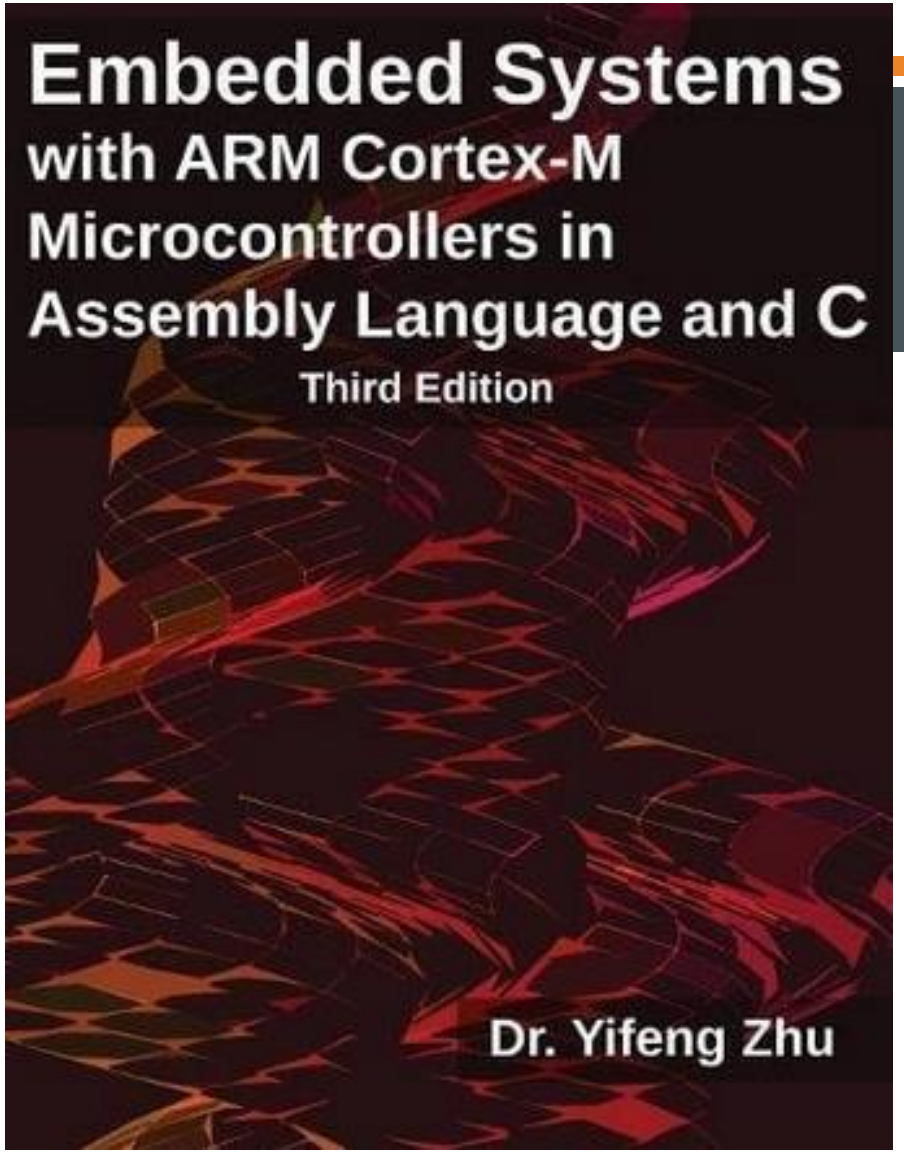
PRIMASK FAULTMASK BASEPRI: 0000 FPCA SPSEL nPRIV

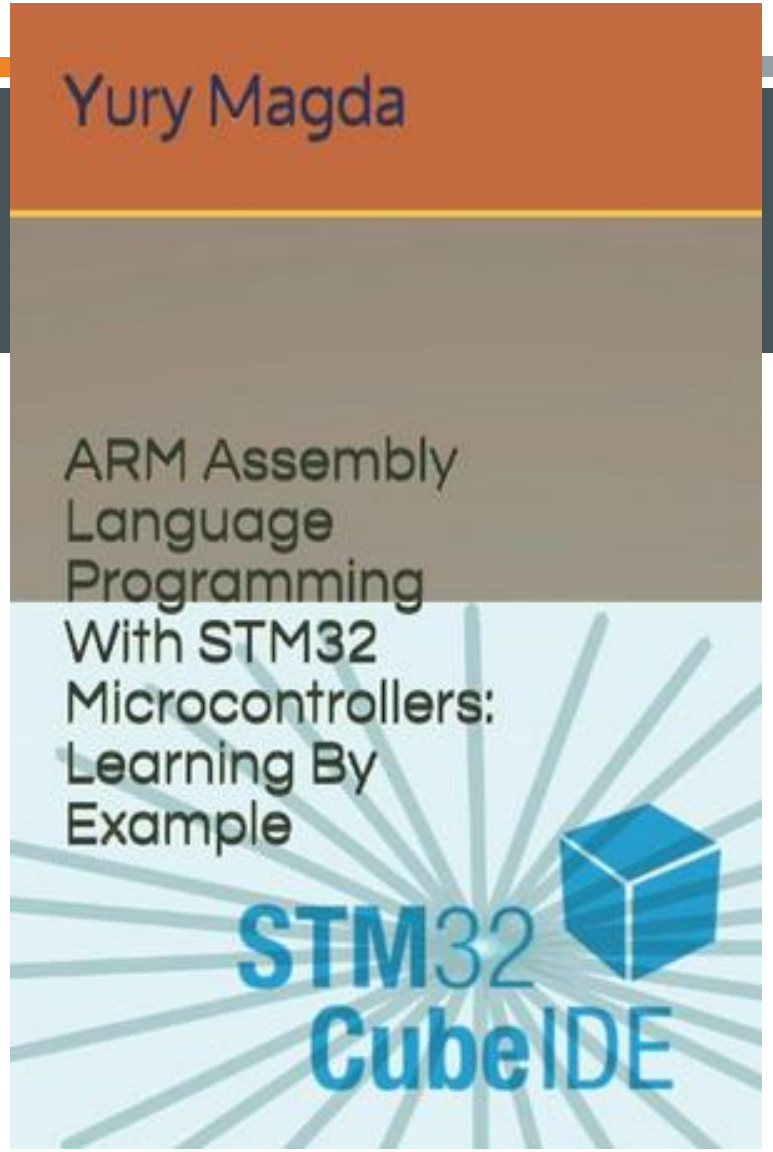
Точки останова: Удалить

Адрес	Описание
0x0800004c	ldr r0 , [pc , #56] ; (8000088 < MAIN_LOOP + 0x16 >)

Регистры

Регистр	Бит	HEX	BIN	DEC
r0	32	0x00000065	0b 0000 0000 0000 0000 0000 0000 0110 0101	101
r1	32	0x20000000	0b 0010 0000 0000 0000 0000 0000 0000 0000	536870912
r2	32	0xe000e010	0b 1110 0000 0000 0000 1110 0000 0001 0000	3758153744
r3	32	0x000000f0	0b 0000 0000 0000 0000 0000 0000 1111 0000	240
r4	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r5	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r6	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r7	32	0x40023800	0b 0100 0000 0000 0010 0011 1000 0000 0000	1073887232
r8	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r9	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r10	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r11	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
r12	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
sp	32	0x2001fff4	0b 0010 0000 0000 0001 1111 1111 1111 0100	537001972
lr	32	0x0800007d	0b 0000 1000 0000 0000 0000 0000 0111 1101	134217853
pc	32	0x080000d0	0b 0000 1000 0000 0000 0000 0000 1101 0000	134217936
xpsr	32	0x21000000	0b 0010 0001 0000 0000 0000 0000 0000 0000	553648128
msp	32	0x2001fff4	0b 0010 0000 0000 0001 1111 1111 1111 0100	537001972
psp	32	0x00000000	0b 0000 0000 0000 0000 0000 0000 0000 0000	0
primask	1	0x00	0b 0000 0000	0
basepri	8	0x00	0b 0000 0000	0
faultmask	1	0x00	0b 0000 0000	0
control	2	0x00	0b 0000 0000	0
d0	64	-->	0x0000000000000000	
d1	64	-->	0x0000000000000000	


Embedded Systems
with ARM Cortex-M
Microcontrollers in
Assembly Language and C
Third Edition
Dr. Yifeng Zhu


Yury Magda
ARM Assembly
Language
Programming
With STM32
Microcontrollers:
Learning By
Example
STM32
CubeIDE

A

```cpp
void variable_sort_2(int length, int *a)
{
   // Determine the number of elements
   switch (length)
   {
      case 0:
      case 1:
        // Exit if less than 2 elements
        return;
      case 2:
        // Below routine sorts 2 elements
        int tmp = a[0];
        a[0] = (a[1] < a[0]) ? a[1] : a[0];
        a[1] = (a[1] < tmp) ? tmp : a[1];
        return;
   }
}
```

B

```asm
variable_sort_2(int, int*):
        ; Determine the number of elements
        cmp    $2, %edi
        ; Exit if less than 2 elements
        jne     .Label
        ; Below routine sorts 2 elements
        mov    (%rsi), %eax
        mov    4(%rsi), %ecx
        cmp    %eax, %ecx
        mov    %eax, %edx
        cmovl   %ecx, %edx
        mov    %edx, (%rsi)
        cmovg %ecx, %eax
        mov    %eax, 4(%rsi)
.Label:
        retq
```

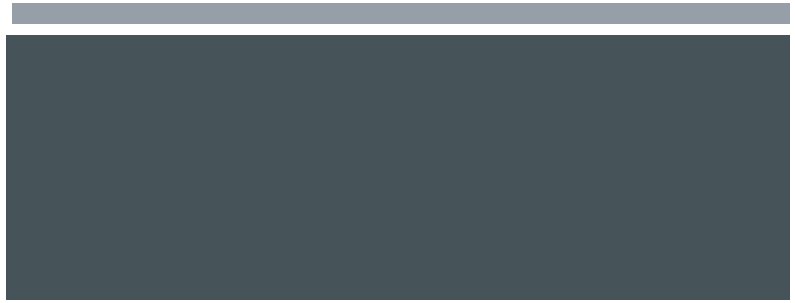
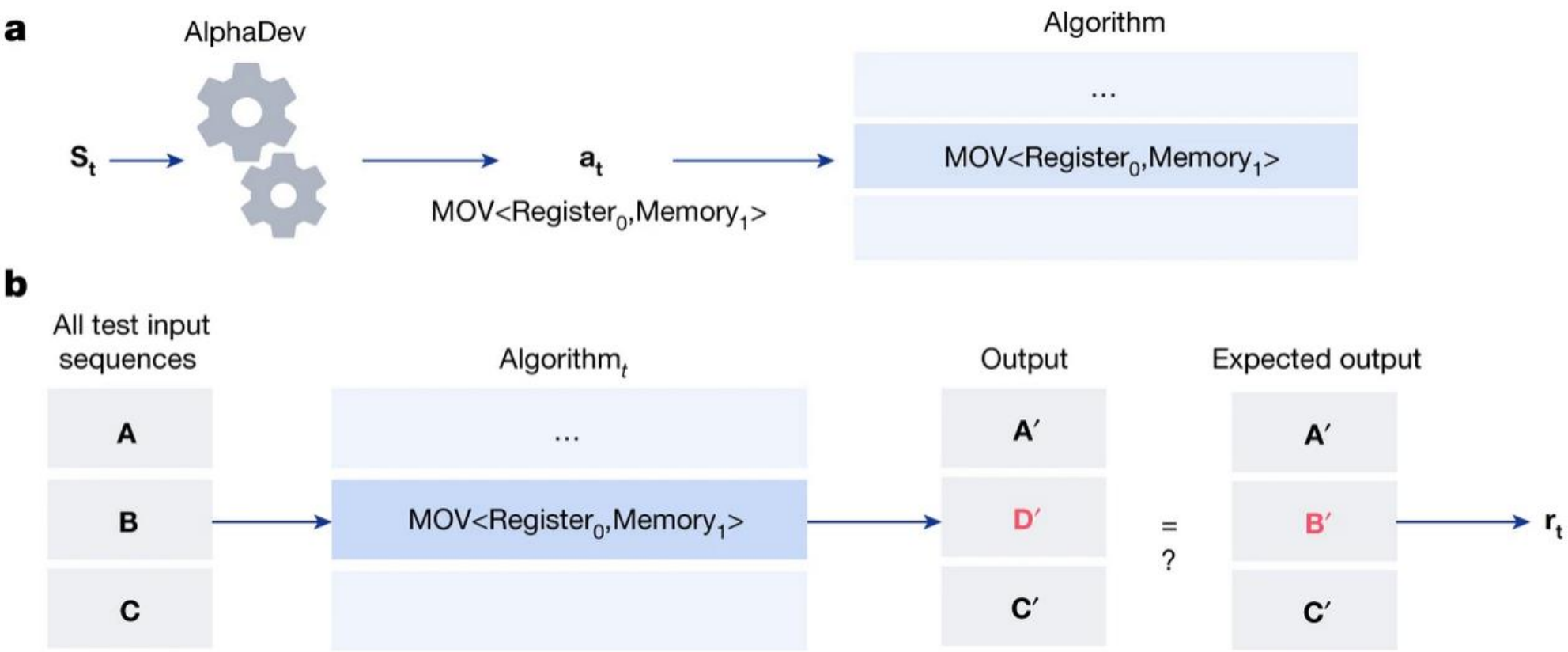


Figure A: An example C++ algorithm that sorts up to two elements.
Figure B: The corresponding assembly representation of the code.

From: Faster sorting algorithms discovered using deep reinforcement learning



a, The AssemblyGame is played by AlphaDev, which receives as input the current assembly algorithm generated thus far $\mathbf{S_t}$ and plays the game by selecting an action to execute. In this example, the action is a mov<Register$_0$,Memory$_1$> assembly instruction, which is appended to the current algorithm. The agent receives a reward that is a function of the algorithm's correctness, discussed in **b**, as well as the algorithm's latency. The game is won by the player discovering a low latency, correct algorithm. **b**, The program correctness and latency computations are used to compute the reward r_t. In this example, test sequences are input to the algorithm; for example, in the case of sorting three elements, test inputs comprise all sequences of unsorted elements of length 3. For each sequence, the algorithm output is compared to the expected output (in the case of sorting, the expected output is the sorted elements). In this example, the output $\mathbf{D}\prime$ does not match the expected output $\mathbf{B}\prime$ and the algorithm is therefore incorrect.

https://www.nature.com/articles/s41586-023-06004-9

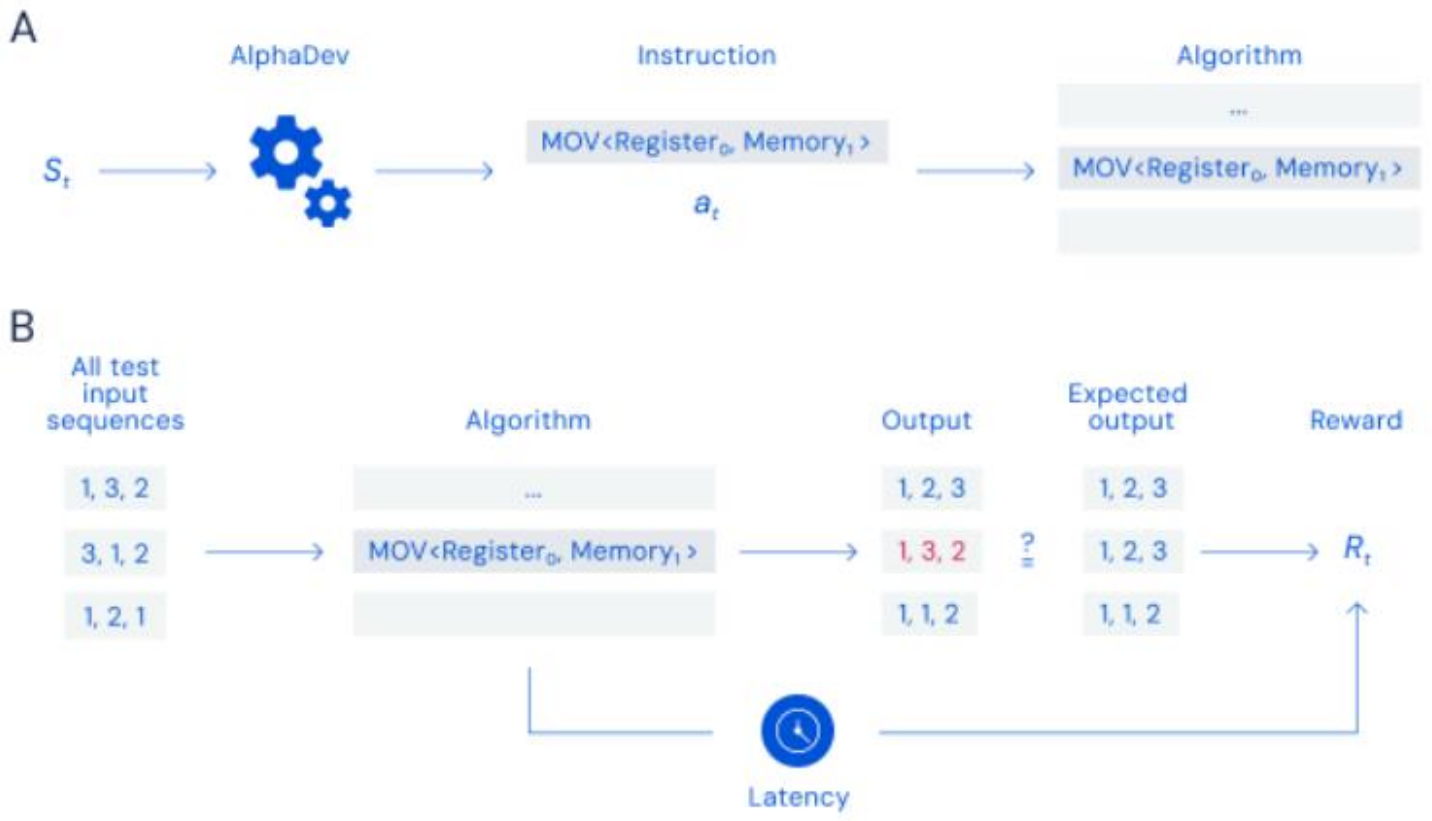
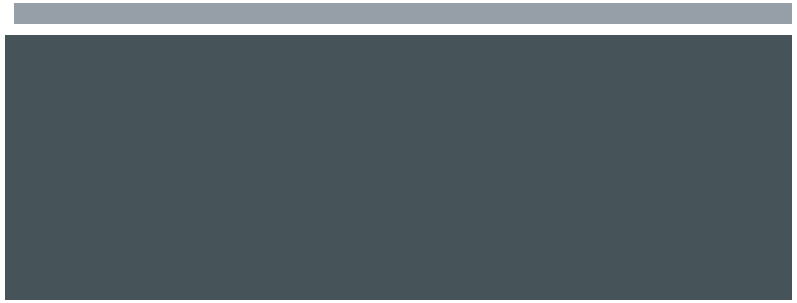


Figure A: The assembly game. The player, AlphaDev, receives the state of the system st as input and plays a move at by selecting an assembly instruction to add to the algorithm that has been generated thus far.

Figure B: The reward computation. After each move, the generated algorithm is fed test input sequences - for sort3, this corresponds to all combinations of sequences of three elements. The algorithm then generates an output, which is compared to the expected output of sorted sequences for the case of sorting. The agent is rewarded based on the algorithm's correctness and latency.

Matrix multiplication

Matrix multiplication is one of the simplest operations in algebra, commonly taught in high school maths classes. But outside the classroom, this humble mathematical operation has enormous influence in the contemporary digital world and is ubiquitous in modern computing.



$$\begin{bmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \\ 5 & -1 & 2 \end{bmatrix} \times \begin{bmatrix} 2 & -1 & 0 \\ 5 & 1 & -1 \\ -2 & 0 & 0 \end{bmatrix} = \begin{bmatrix} -2 & -1 & 0 \\ 11 & -2 & -1 \\ 1 & -6 & 1 \end{bmatrix}$$

$$3 \times 2 + 1 \times 5 + 0 \times -2 = 11$$

Example of the process of multiplying two 3x3 matrices.

Standard algorithm

$$h_1 = a_{1,1}\ b_{1,1}$$
$$h_2 = a_{1,1}\ b_{1,2}$$
$$h_3 = a_{1,2}\ b_{2,1}$$
$$h_4 = a_{1,2}\ b_{2,2}$$
$$h_5 = a_{2,1}\ b_{1,1}$$
$$h_6 = a_{2,1}\ b_{1,2}$$
$$h_7 = a_{2,2}\ b_{2,1}$$
$$h_8 = a_{2,2}\ b_{2,2}$$

$$c_{1,1} = h_1 + h_3$$
$$c_{1,2} = h_2 + h_4$$
$$c_{2,1} = h_5 + h_7$$
$$c_{2,2} = h_6 + h_8$$

Strassen's algorithm

$$h_1 = (a_{1,1} + a_{2,2})\ (b_{1,1} + b_{2,2})$$
$$h_2 = (a_{2,1} + a_{2,2})\ b_{1,1}$$
$$h_3 = a_{1,1}\ (b_{1,2} - b_{2,2})$$
$$h_4 = a_{2,2}\ (-b_{1,1} + b_{2,1})$$
$$h_5 = (a_{1,1} + a_{1,2})\ b_{2,2}$$
$$h_6 = (-a_{1,1} + a_{2,1})\ (b_{1,1} + b_{1,2})$$
$$h_7 = (a_{1,2} - a_{2,2})(b_{2,1} + b_{2,2})$$

$$c_{1,1} = h_1 + h_4 - h_5 + h_7$$
$$c_{1,2} = h_3 + h_5$$
$$c_{2,1} = h_2 + h_4$$
$$c_{2,2} = h_1 - h_2 + h_3 + h_6$$

Standard algorithm

$$h_1 = a_{1,1}\ b_{1,1}$$
$$h_2 = a_{1,1}\ b_{1,2}$$
$$h_3 = a_{1,2}\ b_{2,1}$$
$$h_4 = a_{1,2}\ b_{2,2}$$
$$h_5 = a_{2,1}\ b_{1,1}$$
$$h_6 = a_{2,1}\ b_{1,2}$$
$$h_7 = a_{2,2}\ b_{2,1}$$
$$h_8 = a_{2,2}\ b_{2,2}$$

$$c_{1,1} = h_1 + h_3$$
$$c_{1,2} = h_2 + h_4$$
$$c_{2,1} = h_5 + h_7$$
$$c_{2,2} = h_6 + h_8$$

Strassen's algorithm

$$h_1 = (a_{1,1} + a_{2,2})\ (b_{1,1} + b_{2,2})$$
$$h_2 = (a_{2,1} + a_{2,2})\ b_{1,1}$$
$$h_3 = a_{1,1}\ (b_{1,2} - b_{2,2})$$
$$h_4 = a_{2,2}\ (-b_{1,1} + b_{2,1})$$
$$h_5 = (a_{1,1} + a_{1,2})\ b_{2,2}$$
$$h_6 = (-a_{1,1} + a_{2,1})\ (b_{1,1} + b_{1,2})$$
$$h_7 = (a_{1,2} - a_{2,2})(b_{2,1} + b_{2,2})$$

$$c_{1,1} = h_1 + h_4 - h_5 + h_7$$
$$c_{1,2} = h_3 + h_5$$
$$c_{2,1} = h_2 + h_4$$
$$c_{2,2} = h_1 - h_2 + h_3 + h_6$$

Çalışma Soruları

11 karater girilecek 1,2,3 ile 9,10,11 ekrana basılacak diğerlerine «*» koyulacak

59866321573 598*****573

Ekrana bastırılacak

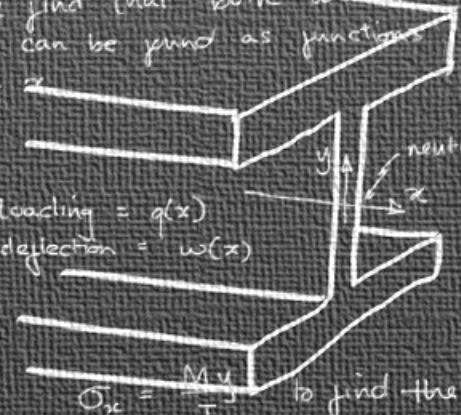
4x4 matris çarması Axw

1,5,3,2,5,4,1,2,3,3,4,5,2,1,3,4,5,1,4,4,2

Dizisinin histogramını bulunuz

Ard arda gelen ikilileri bulunuz. Örn. «5,5» , «3,3»

given a beam of constant EI
we find that both w and
q can be found as functions
of x



loading = $q(x)$
deflection = $w(x)$

$\sigma_x = \frac{My}{I}$ to find the tension force at some location y

for a static system

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = q$$

for $\frac{\partial EI(x)}{\partial x} = 0$

where $\frac{\partial^4 w}{\partial x^4} EI = q(x)$

E = Young's Modulus

I = Second moment of area

http://bsnscb.com/data/out/124/39800806-maths-wallpapers.jpg